

École Supérieure Privée de Management de Tunis
Esprit School of Business

RAPPORT DE STAGE

En vue de l'obtention du diplôme de
Licence en Business Computing
Parcours **Business Intelligence**

*« Conception et développement d'un système intelligent de gestion des rôles
et de pointage des employés avec tableau de bord décisionnel »*

Du 16/02/2026 au 16/06/2026



Réalisé par :
Oussema Korbosli

Soutenu le :

Maître de stage :
Melek Aziz Elmoula

Encadrant académique :
Heni Abidi

Année universitaire 2025/2026

Résumé

Réalisé au sein de la **BTK – Banque Tuniso-Koweïtienne**, ce projet de fin d'études porte sur la conception et le développement d'un **système intelligent de gestion des rôles et de pointage des employés**, doté d'un tableau de bord décisionnel (Business Intelligence). L'application centralise la gestion des agences, des employés, des clients et des objectifs commerciaux, gère les rôles et la sécurité des accès, assure le **pointage (présence) des employés**, et formalise les opérations sensibles au moyen de circuits de validation. Développée avec la plateforme **Jakarta EE 10** (Servlets, JSP, EJB, JPA/Hibernate) au-dessus d'une base **Oracle**, elle est complétée par un volet décisionnel : des vues analytiques exposées à **Microsoft Power BI** et un tableau de bord embarqué restituant les indicateurs clés (effectifs, présence, production). La solution améliore la centralisation des données, la traçabilité des opérations et l'aide à la décision.

Mots-clés : gestion des rôles, pointage des employés, Business Intelligence, aide à la décision, Jakarta EE, JPA, Oracle, Power BI, tableau de bord.

Abstract

Carried out within **BTK – Banque Tuniso-Koweïtienne**, this end-of-studies project deals with the design and development of an **intelligent system for managing employee roles and time-tracking (attendance)**, featuring a business-intelligence dashboard. The application centralizes the management of branches, employees, customers and commercial objectives, handles roles and access security, records **employee attendance (clock-in/clock-out)**, and formalizes sensitive operations through approval workflows. Built on the **Jakarta EE 10** platform (Servlets, JSP, EJB, JPA/Hibernate) over an **Oracle** database, it is complemented by a decision-support layer : analytical views exposed to **Microsoft Power BI** and an embedded dashboard displaying key indicators (headcount, attendance, production). The solution improves data centralization, operation traceability and decision-making.

Key words : role management, employee time-tracking, Business Intelligence, decision support, Jakarta EE, JPA, Oracle, Power BI, dashboard.

Remerciements

Au terme de ce projet de fin d'études, je tiens à exprimer ma profonde gratitude à toutes les personnes qui ont contribué, de près ou de loin, à la réussite de ce travail.

Je remercie chaleureusement mon encadrant académique, **Heni Abidi**, pour ses conseils avisés, sa disponibilité et son suivi rigoureux tout au long de ce projet.

Mes remerciements s'adressent également à mon maître de stage, **Melek Aziz Elmoula**, Data Scientist au sein de la **BTK – Banque Tuniso-Koweïtienne**, pour la confiance accordée, l'accompagnement et le partage de l'expérience métier.

Je tiens aussi à remercier l'ensemble du corps enseignant et administratif de l'**Esprit School of Business** pour la qualité de la formation dispensée durant mon cursus en Licence Business Computing, parcours Business Intelligence.

Enfin, j'exprime ma reconnaissance à ma famille et à mes amis pour leur soutien constant et leurs encouragements.

Table des matières

Remerciements	i
Liste des acronymes	vi
Introduction générale	1
1 Contexte Général	2
1.1 Présentation de l'organisme d'accueil	2
1.1.1 Missions	2
1.1.2 Objectifs	3
1.2 Cadre du projet	3
1.2.1 Analyse de la situation existante	3
1.2.2 Critique de la situation existante	3
1.2.3 Problématique	4
1.2.4 Solutions proposées	4
1.3 Cahier des charges	4
1.3.1 Besoins fonctionnels	4
1.3.2 Besoins non fonctionnels	5
1.4 Méthodologie de gestion de projet	5
1.4.1 Comparaison entre les méthodes agiles et traditionnelles	5
1.4.2 Choix de l'approche	5
1.4.3 Comparaison entre méthodes agiles	6
1.4.3.1 Modèle Kanban	6
1.4.3.2 Modèle Scrum	6
1.4.3.3 Choix de la méthodologie	6
1.5 Mise en œuvre du projet avec Scrum	6
1.5.1 Rôles Scrum	6
1.5.2 Backlog produit	7
1.5.3 Plan des Sprints Agile Scrum	7
1.6 Environnement de travail et ressources	8
1.6.1 Environnement matériel	8
1.6.2 Environnement logiciel et outils	8
2 Analyse et spécification des besoins	10
2.1 Identification des acteurs	10
2.2 Diagramme de cas d'utilisation global	10
2.3 Description de quelques cas d'utilisation	12
2.3.1 Cas « S'authentifier »	12

2.3.2	Cas « Soumettre / valider une demande »	12
2.3.3	Cas « Pointer la présence »	12
3	Conception	13
3.1	Architecture générale	13
3.2	Architecture logique	13
3.3	Architecture physique	14
3.4	Conception détaillée	14
3.4.1	Diagramme de classes	14
3.4.2	Diagramme de séquence : authentification	16
3.4.3	Diagramme de séquence : validation d'une demande	16
3.4.4	Diagramme d'états-transitions : cycle de vie d'une demande	17
3.4.5	Diagramme de séquence : pointage d'un employé	18
3.4.6	Diagramme d'activité : traitement d'une demande	18
3.5	Conception de la base de données	20
3.6	Conception du volet décisionnel	20
4	Réalisation	22
4.1	Choix technologiques	22
4.2	Architecture de déploiement	22
4.3	Présentation des interfaces	22
4.3.1	Page d'authentification	22
4.3.2	Tableau de bord	23
4.3.3	Modules de gestion et de demandes	23
4.4	Réalisation du volet décisionnel	24
4.5	Module de pointage	24
4.6	Sécurité et gestion des rôles	24
	Conclusion générale et perspectives	26
	Nétographie	27
A	Annexes	28
A.1	Scripts SQL d'initialisation	28
A.2	Captures complémentaires	28
A.3	Dépôt du code source	28

Table des figures

1.1	Logo de la BTK – Banque Tuniso-Koweïtienne	2
1.2	Planification du projet (diagramme de Gantt des sprints)	7
2.1	Diagramme de cas d'utilisation global	11
3.1	Architecture logique en couches de l'application	14
3.2	Architecture physique (diagramme de déploiement 3-tiers)	15
3.3	Diagramme de classes du modèle métier	15
3.4	Diagramme de séquence – Authentification	16
3.5	Diagramme de séquence – Validation d'une demande	17
3.6	Diagramme d'états-transitions – Cycle de vie d'une demande	17
3.7	Diagramme de séquence – Pointage d'un employé	18
3.8	Diagramme d'activité – Traitement d'une demande	19
3.9	Schéma relationnel de la base de données	20
3.10	Modèle décisionnel en étoile (vues V_BI_*)	20
4.1	Interface d'authentification	23
4.2	Tableau de bord de l'application	23
4.3	Modules de gestion et de validation des demandes	23
4.4	Rapport Power BI – analyse des effectifs et des objectifs	24
4.5	Interface du module de pointage	24

Liste des tableaux

1.1	Fiche signalétique de la BTK	3
1.2	Besoins non fonctionnels	5
1.3	Méthodes traditionnelles vs méthodes agiles	5
1.4	Kanban vs Scrum	6
1.5	Répartition des rôles Scrum	7
1.6	Extrait du backlog produit	7
1.7	Plan des sprints	8
1.8	Environnement logiciel et outils	8
2.1	Identification des acteurs	10
3.1	Indicateurs de performance (KPI) du volet décisionnel	21

Liste des acronymes

Acronyme	Signification
BTK	Banque Tuniso-Koweïtienne
BI	Business Intelligence (informatique décisionnelle)
KPI	Key Performance Indicator (indicateur clé de performance)
PFE	Projet de Fin d'Études
UML	Unified Modeling Language
JPA	Jakarta Persistence API
EJB	Enterprise Java Beans
JSP	Jakarta Server Pages
JSTL	Jakarta Standard Tag Library
JTA	Jakarta Transaction API
JDBC	Java Database Connectivity
MVC	Modèle–Vue–Contrôleur
SGBD	Système de Gestion de Base de Données
WAR	Web Application Archive
CRUD	Create, Read, Update, Delete
HTTP	HyperText Transfer Protocol

Introduction générale

La transformation numérique du secteur bancaire impose aux établissements de disposer de systèmes d'information capables, d'une part, de centraliser la gestion de leur réseau d'agences et, d'autre part, d'exploiter leurs données pour piloter l'activité. La maîtrise de l'information — effectifs, portefeuille clients, objectifs commerciaux — constitue aujourd'hui un levier déterminant de performance et d'aide à la décision.

C'est dans ce contexte que s'inscrit ce projet de fin d'études, réalisé au sein de la **BTK – Banque Tuniso-Koweïtienne** en vue de l'obtention de la Licence en Business Computing, parcours Business Intelligence, à l'Esprit School of Business. Le projet consiste à concevoir et réaliser une application web de gestion des agences bancaires, adossée à un volet décisionnel destiné à l'aide à la prise de décision.

L'application couvre la gestion des agences, des employés, des clients et des objectifs commerciaux, le **pointage (présence) des employés**, ainsi qu'un ensemble de circuits de validation (workflows) encadrant la création et la modification des données sensibles. À ces fonctionnalités transactionnelles s'ajoute un volet décisionnel : des vues analytiques exposées à Microsoft Power BI et un tableau de bord embarqué offrant aux responsables une vision synthétique de l'activité.

Ce rapport est organisé en quatre chapitres. Le **premier** présente le cadre général du projet : l'organisme d'accueil, l'étude de l'existant, la problématique, la solution proposée et la méthodologie de travail. Le **deuxième** est consacré à l'analyse et à la spécification des besoins. Le **troisième** détaille la conception de l'application et de son volet décisionnel, en s'appuyant sur les architectures logique et physique et sur les diagrammes UML. Le **quatrième** décrit la réalisation, l'environnement et les technologies mises en œuvre. Une conclusion générale dresse le bilan du travail et ouvre des perspectives d'évolution.

Chapitre 1

Contexte Général

Introduction

Ce premier chapitre situe le projet dans son contexte global. Il présente l'organisme d'accueil et ses missions, délimite le cadre du projet à travers l'étude de l'existant et la problématique, formalise le cahier des charges, justifie la méthodologie de gestion de projet retenue, détaille sa mise en œuvre selon le cadre Scrum, et décrit enfin l'environnement de travail.

1.1 Présentation de l'organisme d'accueil

La **Banque Tuniso-Koweïtienne** (BTK Bank) est une banque universelle de droit tunisien, créée le 25 février 1981 dans le cadre d'une convention de coopération bilatérale entre la Tunisie et le Koweït. Constituée en société anonyme et régie par la loi bancaire n° 2016-48, elle dispose d'un capital social de 200 millions de dinars et a son siège social à Tunis. À travers un réseau d'agences couvrant l'ensemble du territoire tunisien, elle accompagne aussi bien les particuliers que les entreprises.



FIGURE 1.1 – Logo de la BTK – Banque Tuniso-Koweïtienne

Le tableau 1.1 en résume la fiche signalétique.

1.1.1 Missions

En tant que banque universelle, la BTK assure un ensemble de missions couvrant les principaux métiers bancaires :

- **Banque de détail** : comptes, cartes, crédits à la consommation et immobiliers, épargne et banque digitale destinés aux particuliers ;
- **Banque des entreprises** : financement de l'investissement et du fonctionnement, financement du commerce extérieur et gestion de trésorerie ;

TABLE 1.1 – Fiche signalétique de la BTK

Élément	Renseignement
Dénomination sociale	Banque Tuniso-Koweïtienne (BTK Bank)
Forme juridique	Société Anonyme (S.A.)
Date de création	25 février 1981
Capital social	200 000 000 DT
Siège social	10 bis, Avenue Mohamed V, 1001 Tunis
Secteur d'activité	Banque universelle
Actionnaire de référence	Groupe Elloumi (60 %)
Réseau	5 directions régionales, 6 succursales, 45 agences
Effectif (2024)	≈ 400 collaborateurs

- **Opérations internationales et de marché** : change, couverture du risque de change et placements ;
- **Innovation et digitalisation** : modernisation des services et offre en ligne.

1.1.2 Objectifs

À travers ces missions, la BTK poursuit plusieurs objectifs stratégiques :

- soutenir le tissu économique en finançant les entreprises et les projets d'investissement ;
- proposer des solutions financières adaptées aux ménages ;
- contribuer à la création d'emplois et à la dynamique de croissance nationale ;
- consolider sa solidité financière et accélérer sa transformation digitale.

1.2 Cadre du projet

Le projet s'inscrit dans la gestion du réseau d'agences de la banque. Il a été réalisé durant le stage de fin d'études, du **16 février au 16 juin 2026**.

1.2.1 Analyse de la situation existante

La gestion actuelle du réseau repose en grande partie sur des traitements manuels et des fichiers bureautiques non centralisés. Les informations relatives aux agences, aux employés et aux clients sont saisies et conservées de manière hétérogène ; les demandes de création ou de modification circulent par des canaux informels ; et la présence des employés n'est suivie par aucun dispositif formalisé.

1.2.2 Critique de la situation existante

Cette organisation présente plusieurs limites :

- dispersion et redondance des données, sources d'incohérences ;
- absence de circuit de validation formalisé pour les opérations sensibles ;
- absence de suivi de présence (pointage) des employés ;
- faible traçabilité des actions et des décisions ;
- difficulté à produire des indicateurs consolidés pour le pilotage ;
- gestion des droits d'accès insuffisamment maîtrisée.

1.2.3 Problématique

Comment **centraliser et fiabiliser** la gestion du réseau d'agences bancaires, **encadrer** les opérations sensibles par des circuits de validation, **assurer le suivi de présence** des employés, et **exploiter** les données ainsi structurées pour fournir aux responsables des indicateurs d'aide à la décision ?

1.2.4 Solutions proposées

La solution retenue consiste en une application web centralisée, structurée autour d'une base de données Oracle, qui couvre la gestion des agences, des employés, des clients et des objectifs, gère les rôles et la sécurité des accès, assure le **pointage** de présence des employés, et formalise les demandes de création et de modification au moyen de workflows de validation. Un **volet décisionnel** complète le dispositif : des vues analytiques alimentent Microsoft Power BI, et un tableau de bord embarqué restitue les indicateurs clés au sein de l'application.

1.3 Cahier des charges

Le cahier des charges formalise les besoins auxquels la solution doit répondre.

1.3.1 Besoins fonctionnels

Le système doit permettre :

- l'authentification sécurisée et la gestion des rôles et des droits d'accès (ADMIN, DIRECTEUR_COMMERCIAL, USER) ;
- la gestion (CRUD) des agences, des employés et des clients ;
- la consultation des objectifs commerciaux et le suivi de la production ;
- le **pointage (présence) des employés** : saisie de l'heure d'arrivée et de départ, statut (présent / retard / absent), en mode automatique (bouton) ou manuel (administrateur) ;
- la soumission et la validation des demandes (clients, employés, modifications) selon le cycle EN_ATTENTE → EN_COURS → EFFECTUE/REFUSE ;
- la notification des demandes en attente et la journalisation des actions ;
- la restitution d'indicateurs via un tableau de bord et via Power BI.

1.3.2 Besoins non fonctionnels

Le tableau 1.2 récapitule les besoins non fonctionnels.

TABLE 1.2 – Besoins non fonctionnels

Besoin	Description
Sécurité	Authentification, gestion des rôles et protection des opérations sensibles par des workflows de validation.
Fiabilité	Intégrité des données garantie par le SGBD et les transactions JTA.
Performance	Temps de réponse acceptable sur les volumes manipulés.
Ergonomie	Interface claire et cohérente, navigation par menu latéral.
Maintenabilité	Architecture en couches, code modulaire et faiblement couplé.
Portabilité	Application Jakarta EE standard, déployable sur serveur compatible.

1.4 Méthodologie de gestion de projet

1.4.1 Comparaison entre les méthodes agiles et traditionnelles

Le tableau 1.3 oppose les deux grandes familles de méthodes de conduite de projet.

TABLE 1.3 – Méthodes traditionnelles vs méthodes agiles

Critère	Méthodes traditionnelles	Méthodes agiles
Approche	Séquentielle et planifiée (cascade, cycle en V)	Itérative et incrémentale
Flexibilité	Faible : les changements sont coûteux	Élevée : adaptation continue
Livraison	En fin de projet	Par incréments fréquents
Implication du client	Au début et à la fin	Tout au long du projet
Documentation	Abondante et préalable	Allégée, au juste nécessaire
Détection des risques	Tardive	Précoce

1.4.2 Choix de l'approche

Le périmètre du projet a évolué au fil des échanges avec l'organisme d'accueil et les besoins ont été précisés progressivement. Une **approche agile** a donc été privilégiée : elle permet de livrer la solution par incréments, d'intégrer les retours au fur et à mesure et de maîtriser les risques plus tôt, contrairement à une approche traditionnelle rigide.

1.4.3 Comparaison entre méthodes agiles

Parmi les cadres agiles, deux sont particulièrement répandus : Kanban et Scrum.

1.4.3.1 Modèle Kanban

Kanban repose sur la **visualisation d'un flux continu** de tâches au moyen d'un tableau organisé en colonnes (à faire, en cours, terminé) et sur la limitation du travail en cours. Il n'impose ni rôles ni itérations de durée fixe et convient particulièrement aux activités de flux continu et de maintenance.

1.4.3.2 Modèle Scrum

Scrum organise le travail en **itérations de durée fixe** (les *sprints*), s'appuie sur des **rôles** définis (Product Owner, Scrum Master, équipe de développement), des **artefacts** (*product backlog*, *sprint backlog*) et des **cérémonies** (planification, mêlée quotidienne, revue, rétrospective). Le tableau 1.4 compare les deux cadres.

TABLE 1.4 – Kanban vs Scrum

Critère	Kanban	Scrum
Cadence	Flux continu	Sprints de durée fixe
Rôles	Non imposés	Product Owner, Scrum Master, équipe
Cérémonies	Optionnelles	Planning, mêlée, revue, rétrospective
Gestion du travail	Tableau Kanban, limite du WIP	Sprint backlog
Changements	À tout moment	Figés pendant le sprint
Adapté à	Maintenance, flux continu	Projets à fonctionnalités planifiées

1.4.3.3 Choix de la méthodologie

Le projet visant à développer un ensemble de fonctionnalités planifiées et livrables par incréments, le cadre **Scrum** a été retenu : ses sprints, ses rôles et ses cérémonies offrent une structure adaptée au suivi régulier de l'avancement et à la validation progressive des incréments avec l'organisme d'accueil.

1.5 Mise en œuvre du projet avec Scrum

1.5.1 Rôles Scrum

Le tableau 1.5 présente la répartition des rôles Scrum adoptée durant le projet.

TABLE 1.5 – Répartition des rôles Scrum

Rôle	Assuré par	Responsabilité
Product Owner	Maître de stage (BTK)	Exprime et priorise les besoins (backlog).
Scrum Master	Encadrant / étudiant	Veille au bon déroulement de la démarche.
Équipe de développement	Étudiant	Conçoit, développe et teste les incréments.

1.5.2 Backlog produit

Le *product backlog* regroupe les fonctionnalités à réaliser, exprimées sous forme de *user stories* et priorisées (tableau 1.6).

TABLE 1.6 – Extrait du backlog produit

ID	User story	Priorité
US1	S’authentifier et gérer les rôles et les droits d’accès	Haute
US2	Gérer les agences (créer, consulter, modifier, supprimer)	Haute
US3	Gérer les employés et leur rattachement aux agences	Haute
US4	Gérer les clients et le portefeuille	Moyenne
US5	Suivre les objectifs commerciaux	Moyenne
US6	Pointer la présence (arrivée / départ) des employés	Haute
US7	Soumettre et valider des demandes (workflow)	Moyenne
US8	Consulter le tableau de bord décisionnel	Haute

1.5.3 Plan des Sprints Agile Scrum

Le projet a été découpé en sprints successifs, chacun livrant un incrément fonctionnel (tableau 1.7). La figure 1.2 présente la planification correspondante.

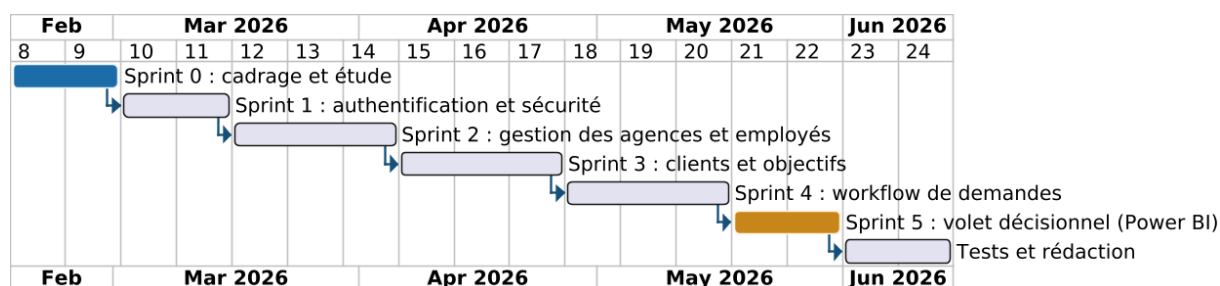


FIGURE 1.2 – Planification du projet (diagramme de Gantt des sprints)

TABLE 1.7 – Plan des sprints

Sprint	Objectif principal
Sprint 0	Cadrage, étude de l'existant et spécification des besoins
Sprint 1	Authentification et gestion des rôles
Sprint 2	Gestion des agences et des employés
Sprint 3	Gestion des clients et des objectifs
Sprint 4	Pointage et workflows de demandes
Sprint 5	Volet décisionnel (vues BI et Power BI)
Sprint 6	Tests, consolidation et rédaction

1.6 Environnement de travail et ressources

1.6.1 Environnement matériel

Le développement a été réalisé sur un poste de travail standard [*à compléter : processeur, mémoire vive, système d'exploitation*].

1.6.2 Environnement logiciel et outils

Le tableau 1.8 récapitule les principaux outils et technologies employés.

TABLE 1.8 – Environnement logiciel et outils

Outil / Technologie	Rôle
JDK 21	Plateforme d'exécution et de compilation Java
Jakarta EE 10	Spécifications d'entreprise (Servlets, JSP, EJB, JPA)
Hibernate	Implémentation JPA (mapping objet-relationnel)
Oracle Database	Système de gestion de base de données
WildFly / JBoss	Serveur d'applications (déploiement du WAR)
Apache Maven	Gestion des dépendances et automatisation du build
IntelliJ IDEA	Environnement de développement intégré
ApexCharts	Graphiques du tableau de bord embarqué
Microsoft Power BI	Restitution et reporting décisionnel
Git / GitHub	Gestion de versions du code source

Conclusion

Ce chapitre a posé le contexte du projet : l'organisme d'accueil et ses missions, l'étude de l'existant et la problématique, le cahier des charges, le choix et la mise en œuvre de la méthodo-

logie Scrum, ainsi que l'environnement de travail. Le chapitre suivant est consacré à l'analyse et à la spécification des besoins à travers la modélisation des cas d'utilisation.

Chapitre 2

Analyse et spécification des besoins

Introduction

Le cahier des charges (besoins fonctionnels et non fonctionnels) ayant été établi au chapitre précédent, ce chapitre approfondit l'analyse par la modélisation des cas d'utilisation : identification des acteurs, diagramme de cas d'utilisation global et description de quelques cas représentatifs.

2.1 Identification des acteurs

Trois profils interagissent avec l'application, conformément aux rôles gérés par le module de sécurité (table SECURITY_USERS). Le tableau 2.1 en précise les responsabilités.

TABLE 2.1 – Identification des acteurs

Acteur	Responsabilités
Administrateur (ADMIN)	Gère les agences, les employés et les clients ; valide les demandes de création de clients et d'employés ; consulte le journal d'audit.
Directeur commercial (DIRECTEUR_COMMERCIAL)	Soumet les demandes de création (clients, employés) ; valide les demandes de modification des données des employés ; consulte les objectifs.
Utilisateur (USER)	Consulte les données autorisées et soumet des demandes de modification de ses propres informations.

2.2 Diagramme de cas d'utilisation global

La figure 2.1 présente le **diagramme de cas d'utilisation global**. Ce diagramme UML a pour objectif de représenter, d'un point de vue fonctionnel, les interactions entre les acteurs et les services offerts par le système.

On y distingue les trois acteurs identifiés et leurs cas d'utilisation respectifs. Le cas *S'authentifier* est commun à tous les acteurs ; il est relié aux autres cas par une relation «include» traduisant le fait que l'accès aux fonctionnalités exige une authentification préalable. L'**administrateur** concentre les cas de gestion (agences, employés, clients), la validation des demandes et la consul-

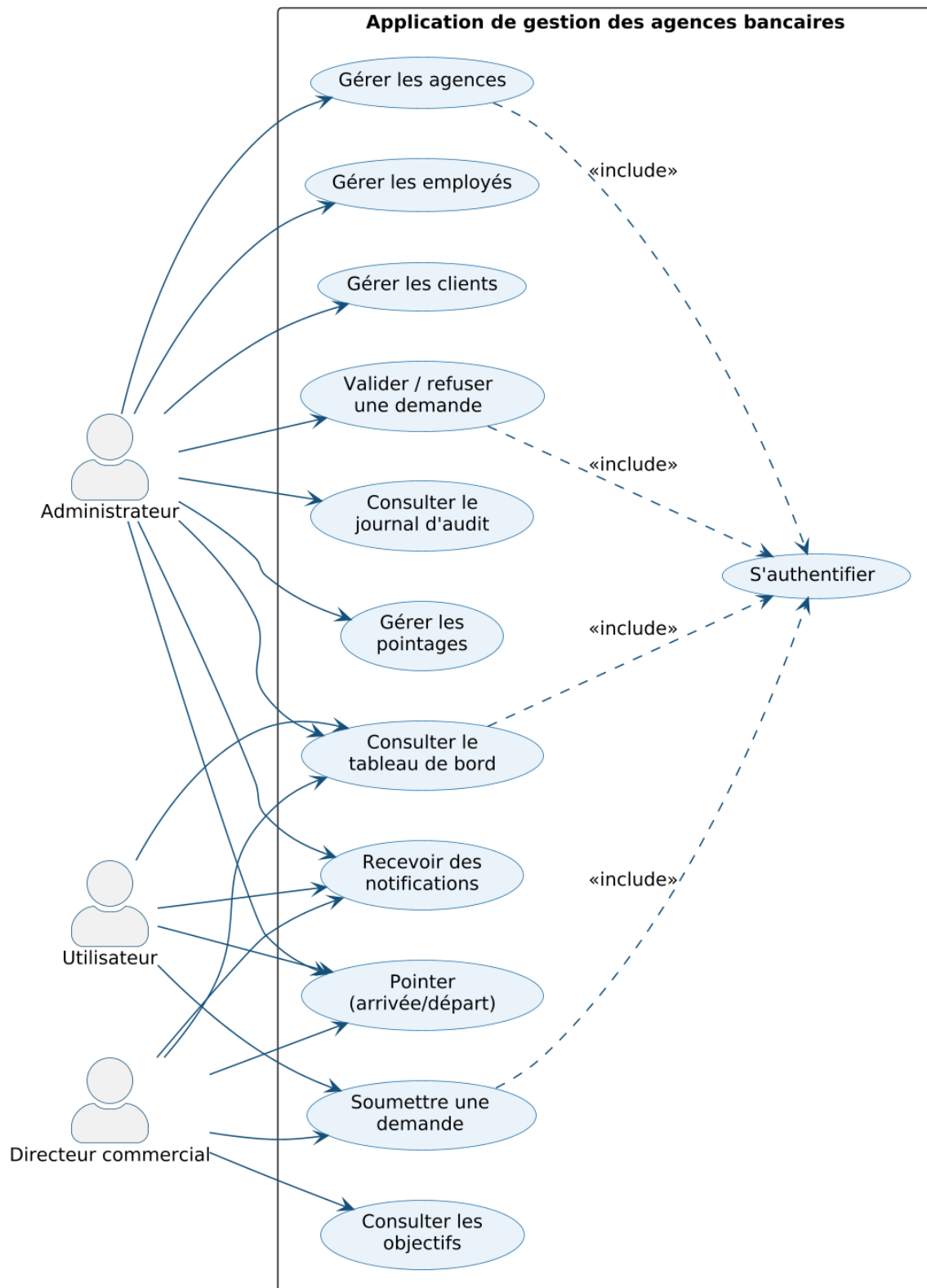


FIGURE 2.1 – Diagramme de cas d'utilisation global

tation du journal ; le **directeur commercial** soumet les demandes et valide les modifications ; l'**utilisateur** se limite à la consultation et à la soumission de demandes le concernant.

2.3 Description de quelques cas d'utilisation

2.3.1 Cas « S'authentifier »

L'utilisateur saisit son identifiant et son mot de passe. Le système vérifie ces informations auprès de la table de sécurité dédiée ; en cas de succès, une session est ouverte et l'utilisateur est redirigé vers le tableau de bord correspondant à son rôle ; sinon, un message d'erreur est affiché.

2.3.2 Cas « Soumettre / valider une demande »

Le directeur commercial soumet une demande (création d'un client, d'un employé ou modification de données), enregistrée avec le statut `EN_ATTENTE` et notifiée à l'administrateur. Ce dernier peut l'approuver ou la refuser en y associant un commentaire ; le statut évolue alors vers `EFFECTUE` ou `REFUSE`, et l'action est journalisée.

2.3.3 Cas « Pointer la présence »

L'employé enregistre son arrivée et son départ en cliquant sur le bouton « Pointer » : le système relève l'heure courante, détermine le statut (`PRESENT`, ou `RETARD` au-delà de l'heure limite) et enregistre le pointage du jour. L'administrateur peut, de son côté, saisir ou corriger manuellement un pointage (`PRESENT`, `RETARD` ou `ABSENT`). Les pointages alimentent ensuite les indicateurs de présence du tableau de bord et du volet décisionnel.

Conclusion

Les besoins étant spécifiés et illustrés, le chapitre suivant présente la conception de l'application et de son volet décisionnel.

Chapitre 3

Conception

Introduction

Ce chapitre décrit la conception de la solution. Il présente successivement l'architecture générale, les architectures **logique** et **physique**, la conception détaillée (diagramme de classes, diagrammes de séquence et d'états-transitions), la conception de la base de données et celle du volet décisionnel.

3.1 Architecture générale

L'application adopte une architecture en couches reposant sur le patron **Modèle–Vue–Contrôleur** (MVC) et la plateforme Jakarta EE. La couche de présentation (JSP/JSTL) restitue les vues ; la couche de contrôle (Servlets) orchestre les requêtes ; la couche métier (EJB) encapsule la logique applicative et les transactions ; la couche de persistance (JPA/Hibernate) assure le mapping objet-relationnel vers la base Oracle.

3.2 Architecture logique

L'architecture logique, présentée à la figure 3.1, décompose le système en **couches** aux responsabilités distinctes et faiblement couplées.

Il s'agit d'un **diagramme de composants** dont l'objectif est de montrer l'organisation interne du logiciel et le flux des appels entre couches :

- **Couche présentation** : pages JSP/JSTL, feuilles de style CSS et tableau de bord Apex-Charts ; elle produit l'interface restituée au navigateur ;
- **Couche contrôle** : Servlets annotés `@WebServlet` et le filtre `NotificationFilter` ; elle reçoit les requêtes HTTP et invoque les services ;
- **Couche métier** : services EJB (`@Singleton`) portant la logique applicative et les transactions ;
- **Couche persistance** : entités JPA et Hibernate, qui dialoguent avec la base Oracle via l'`EntityManager`.

Microsoft Power BI se connecte directement à la base, en lecture, à travers les vues décisionnelles `V_BI_*`.

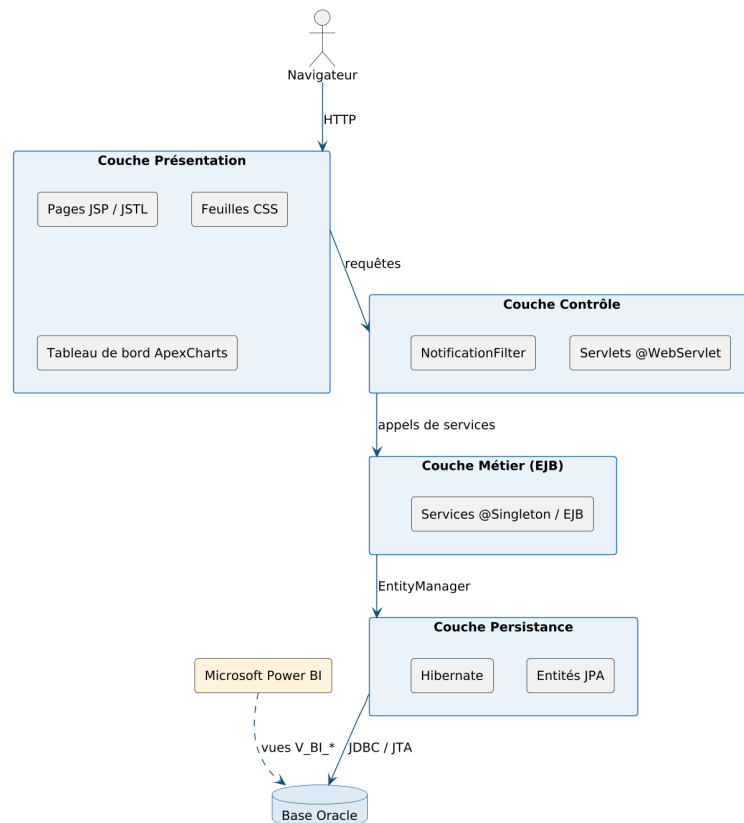


FIGURE 3.1 – Architecture logique en couches de l'application

3.3 Architecture physique

L'architecture physique, illustrée à la figure 3.2, représente le **déploiement** des composants sur les différents nœuds matériels et les liaisons réseau qui les relient.

Ce **diagramme de déploiement** UML décrit une organisation **3-tiers** :

- le **poste client** exécute un navigateur web qui dialogue avec le serveur applicatif en HTTP/HTTPS ;
- le **serveur d'applications** WildFly/JBoss héberge l'artefact déployé `gestion-agences.war` ; il accède à la base via une source de données JTA (OracleDS) en JDBC ;
- le **serveur de base de données** héberge l'instance Oracle (FREEPDB1) ;
- le **poste décisionnel** exécute Power BI Desktop, connecté à la base Oracle pour consommer les vues `V_BI_*`.

3.4 Conception détaillée

3.4.1 Diagramme de classes

La figure 3.3 présente le **diagramme de classes** du modèle métier. Son objectif est de représenter les entités persistantes du domaine, leurs attributs et les associations qui les relient.

L'entité **Utilisateur** occupe une position centrale. Les principales associations, fidèles au modèle réel, sont les suivantes :

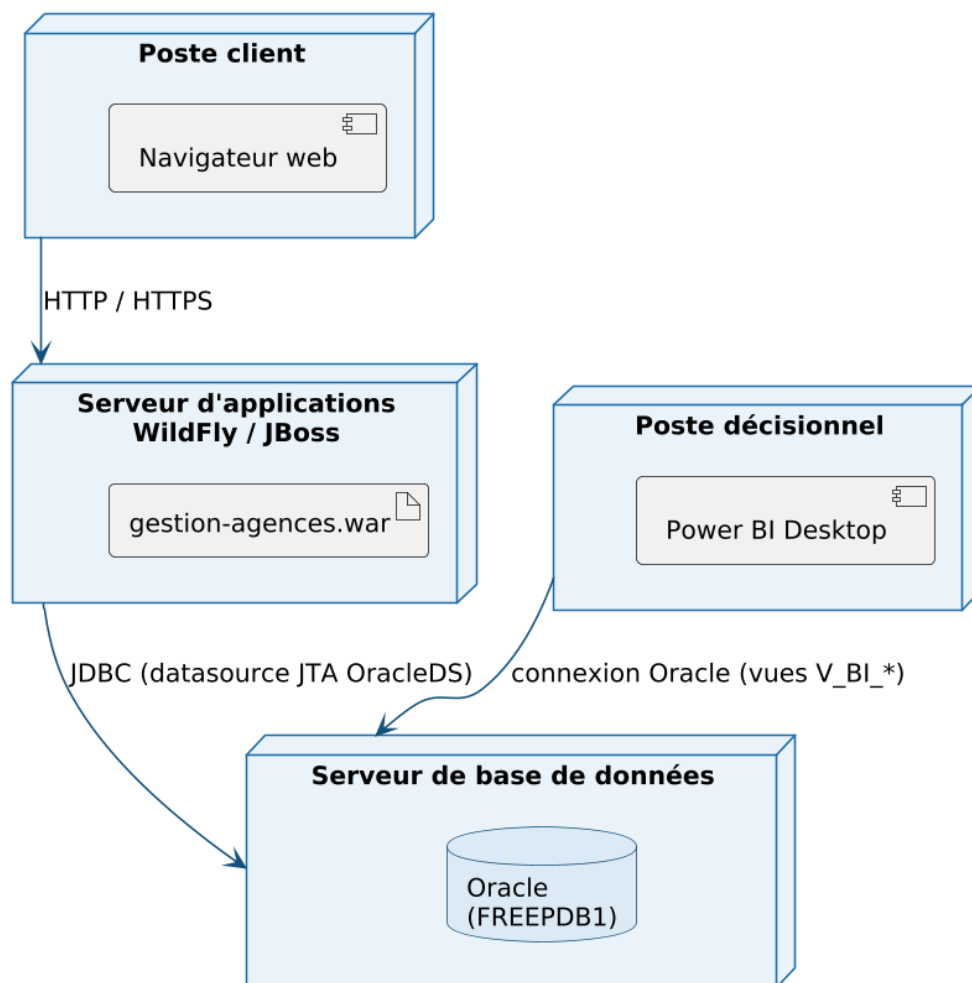


FIGURE 3.2 – Architecture physique (diagramme de déploiement 3-tiers)

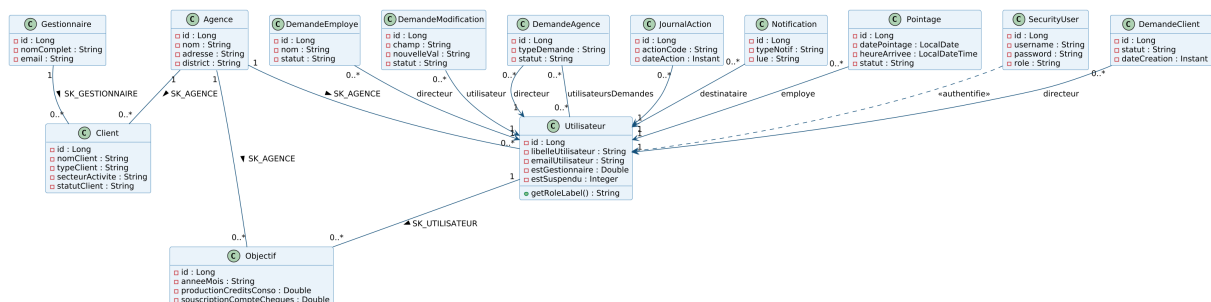


FIGURE 3.3 – Diagramme de classes du modèle métier

- une Agence regroupe plusieurs Utilisateur, Client et Objectif (cardinalité 1–0..*, via la clé étrangère SK_AGENCE);
- un Gestionnaire encadre plusieurs Client (SK_GESTIONNAIRE);
- chaque Objectif est rattaché à un Utilisateur (SK_UTILISATEUR);
- les entités de demande (DemandeClient, DemandeEmploye, DemandeModification, DemandeAgence), ainsi que JournalAction, Notification et Pointage, référencent un Utilisateur par une association @ManyToOne (0..* vers 1);
- DemandeAgence possède en outre une association @ManyToMany avec Utilisateur (utilisateursDem);
- SecurityUser est relié à Utilisateur par une **dépendance** «authentifiée», l'authentification établissant le lien entre identifiant de sécurité et utilisateur métier.

Le modèle ne comporte ni héritage ni composition entre entités : les liens sont exclusivement des associations, dont certaines sont matérialisées par des clés étrangères et d'autres par des relations JPA explicites. Ce choix reflète fidèlement le code du projet.

3.4.2 Diagramme de séquence : authentification

La figure 3.4 décrit, au moyen d'un **diagramme de séquence**, le déroulement dynamique du cas *S'authentifier*.

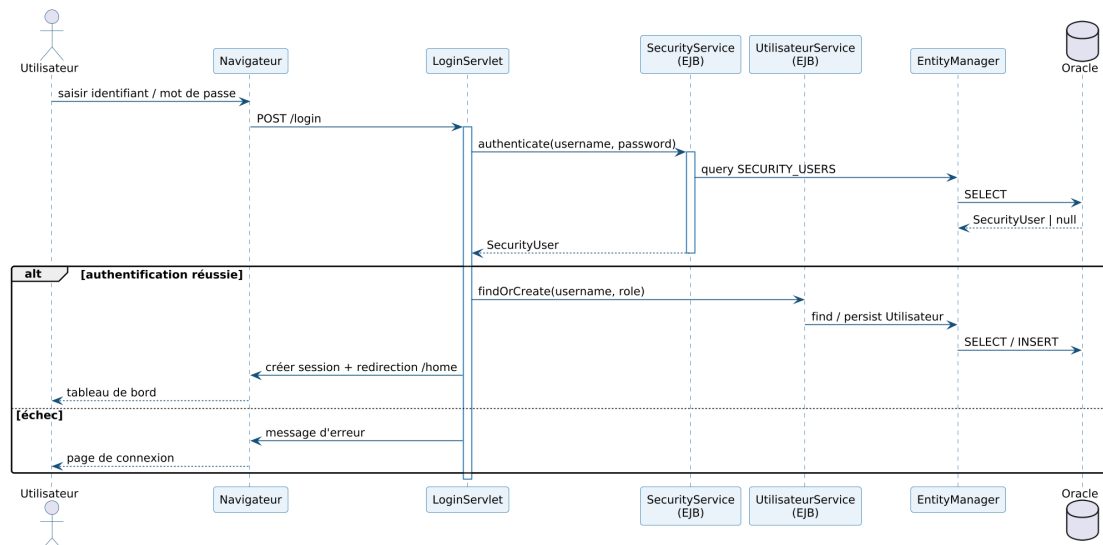


FIGURE 3.4 – Diagramme de séquence – Authentification

Le LoginServlet reçoit la requête POST /login et délègue la vérification au SecurityService, qui interroge la table SECURITY_USERS via l'EntityManager. Le fragment combiné alt distingue les deux issues : en cas de succès, le service UtilisateurService associe (ou crée) l'utilisateur métier, une session est ouverte et l'utilisateur est redirigé vers le tableau de bord ; en cas d'échec, un message d'erreur est renvoyé.

3.4.3 Diagramme de séquence : validation d'une demande

La figure 3.5 détaille le traitement d'une demande de modification par l'administrateur.

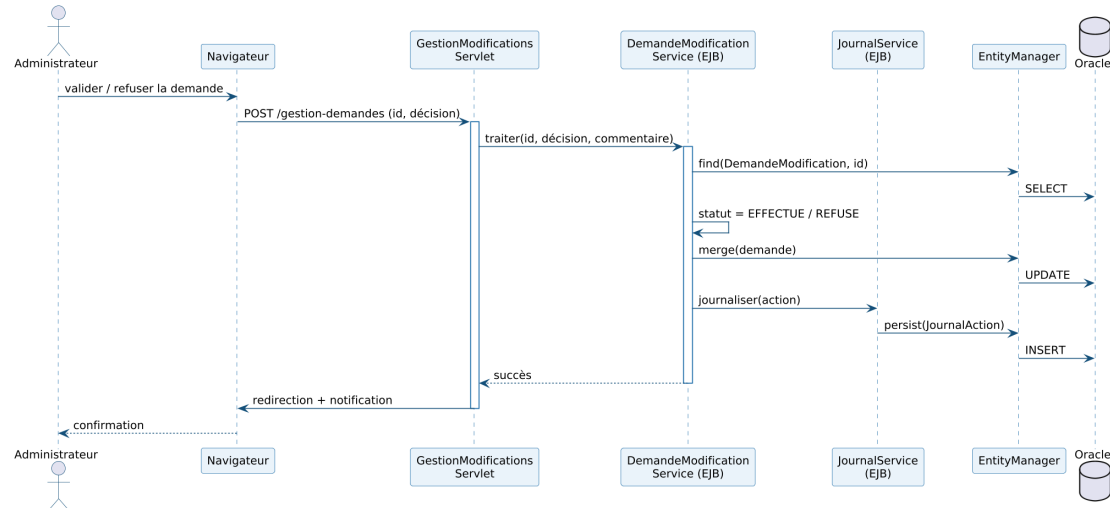


FIGURE 3.5 – Diagramme de séquence – Validation d’une demande

Le `GestionModificationsServlet` transmet la décision au service métier, qui charge la demande, met à jour son statut (EFFECTUE/REFUSE), persiste la modification puis journalise l’action via le `JournalService`. Ce diagramme met en évidence la collaboration entre les couches contrôle, métier et persistance.

3.4.4 Diagramme d’états-transitions : cycle de vie d’une demande

La figure 3.6 modélise, par un **diagramme d’états-transitions**, le cycle de vie d’une demande.

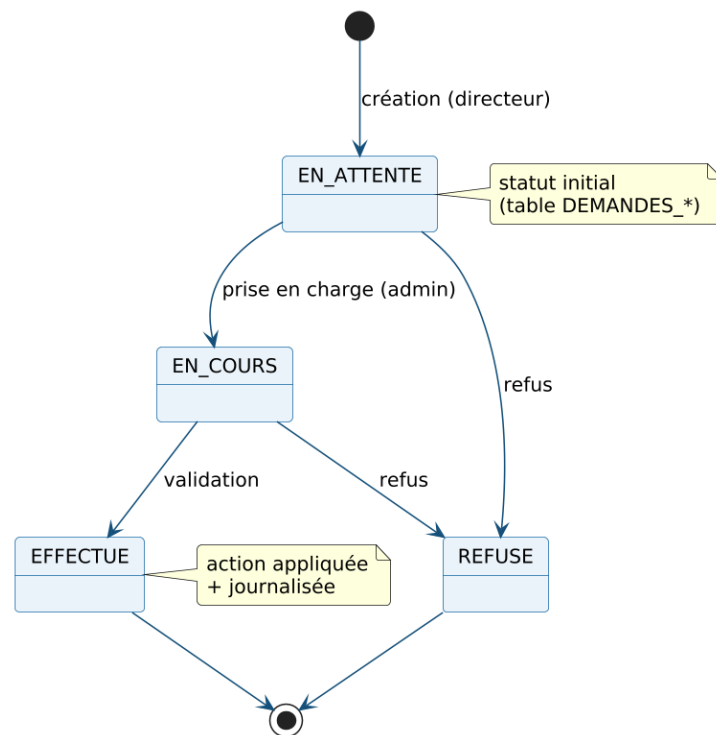


FIGURE 3.6 – Diagramme d’états-transitions – Cycle de vie d’une demande

Une demande naît à l'état EN_ATTENTE, passe en EN_COURS lors de sa prise en charge, puis atteint un état final EFFECTUE ou REFUSE. Ces états correspondent exactement aux valeurs autorisées par la contrainte de la table des demandes, ce qui garantit la cohérence entre le modèle de conception et la base de données.

3.4.5 Diagramme de séquence : pointage d'un employé

La figure 3.7 décrit le scénario de *pointage* d'un employé.

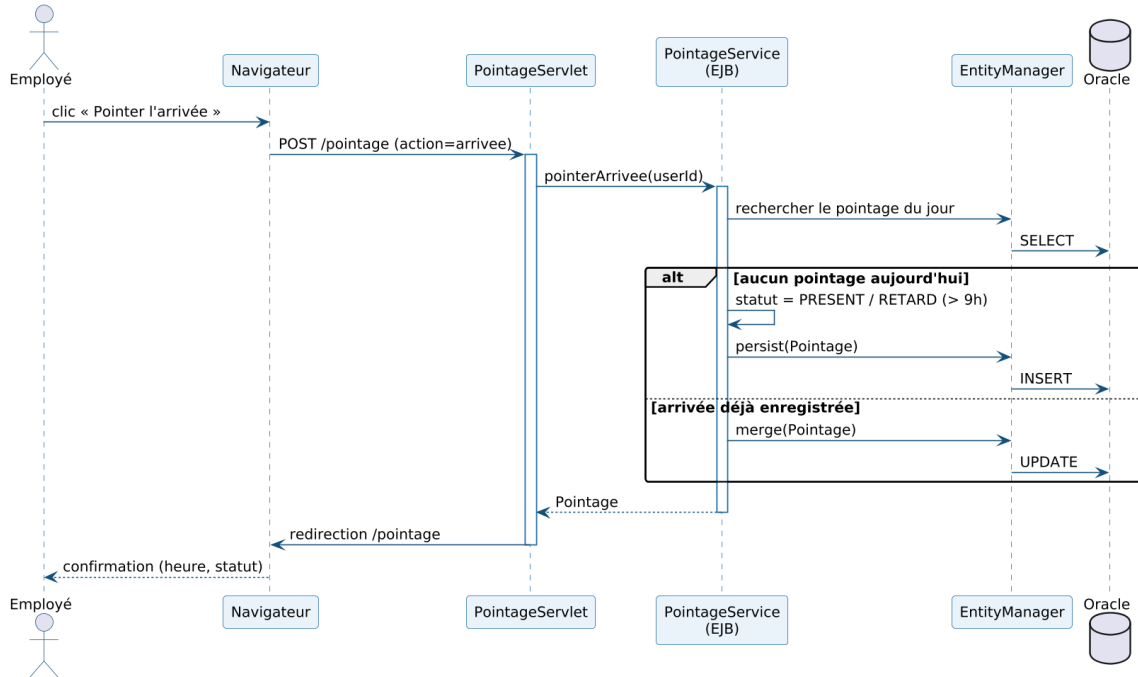


FIGURE 3.7 – Diagramme de séquence – Pointage d'un employé

Le PointageServlet transmet l'action au PointageService, qui recherche le pointage du jour de l'employé. Le fragment combiné alt distingue deux cas : s'il n'existe pas encore de pointage, le service détermine le statut (PRESENT ou RETARD au-delà de 9h) puis persiste un nouvel enregistrement (INSERT) ; sinon, il met à jour l'enregistrement existant (UPDATE). Le résultat est renvoyé à l'employé sous forme de confirmation.

3.4.6 Diagramme d'activité : traitement d'une demande

Au-delà des échanges entre objets, le **diagramme d'activité** de la figure 3.8 représente le *processus métier* de traitement d'une demande, du point de vue du flux de travail.

Le directeur soumet une demande, enregistrée au statut EN_ATTENTE et notifiée à l'administrateur. Le **nœud de décision** oriente le flux selon la décision : en cas de validation, le statut passe à EFFECTUE et la modification est appliquée ; en cas de refus, le statut passe à REFUSE avec un commentaire. Les deux branches convergent vers la journalisation de l'action et la notification du demandeur. Ce diagramme complète le diagramme d'états-transitions en explicitant les *traitements* associés à chaque transition.

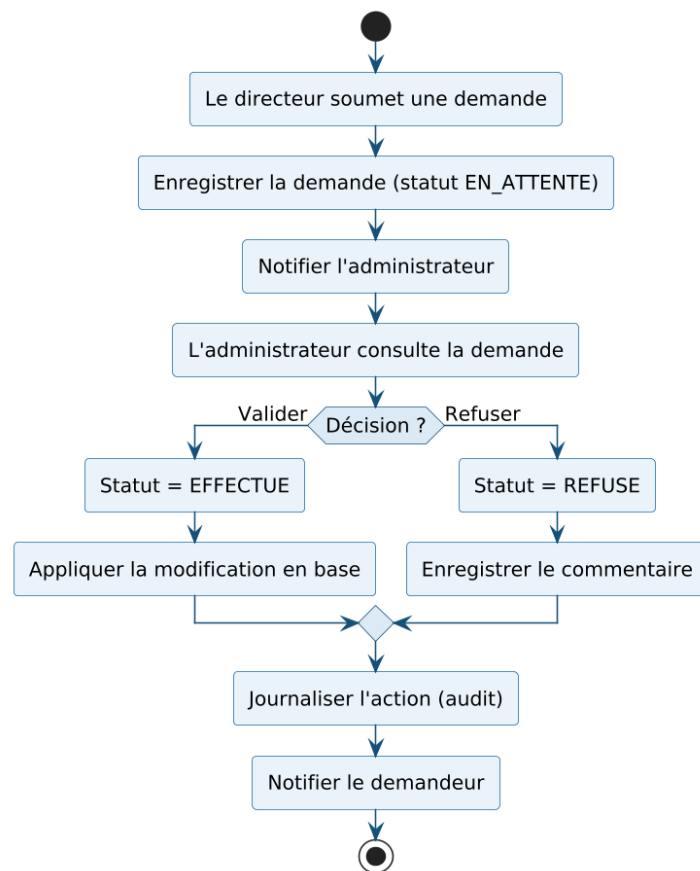


FIGURE 3.8 – Diagramme d'activité – Traitement d'une demande

3.5 Conception de la base de données

La figure 3.9 présente le **schéma relationnel** de la base.

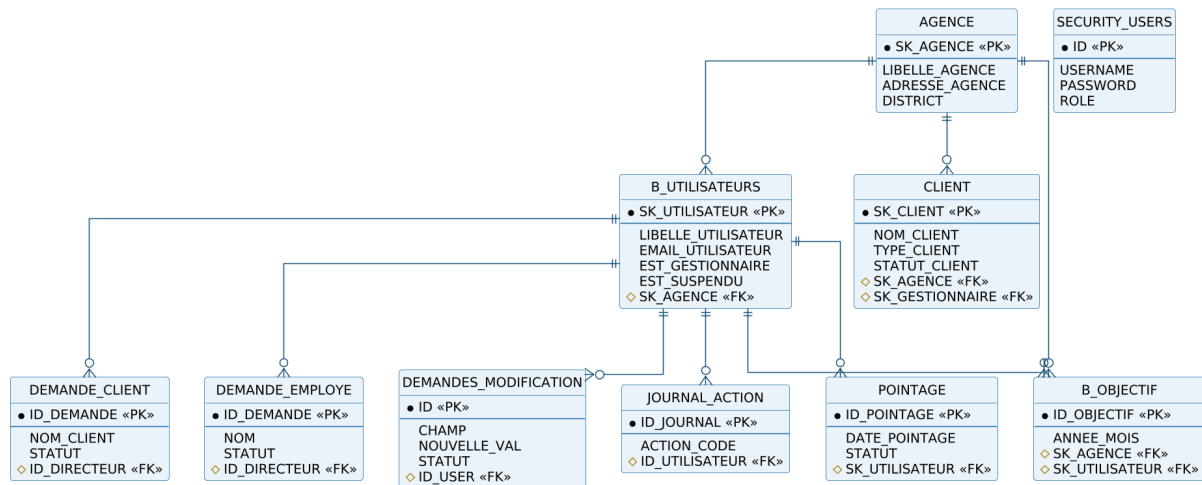


FIGURE 3.9 – Schéma relationnel de la base de données

Le schéma s'articule autour des tables AGENCE, B_UTILISATEURS, CLIENT et B_OBJECTIF, complétées par les tables de sécurité (SECURITY_USERS), de demandes (DEMANDE_CLIENT, DEMANDE_EMPLOYE, DEMANDES_MODIFICATION), de journalisation (JOURNAL_ACTION) et de pointage (POINTAGE). Les clés primaires (PK) et étrangères (FK) ainsi que les cardinalités (notation « patte d'oie ») y sont explicitées.

3.6 Conception du volet décisionnel

La couche décisionnelle transforme les données opérationnelles en informations de pilotage. Quatre vues analytiques jouent le rôle de couche sémantique selon une logique en **étoile** (figure 3.10) : autour de tables de faits (production, effectifs, clients, présence), des dimensions descriptives (agence, gestionnaire, période, type de client) permettent l'analyse multidimensionnelle.

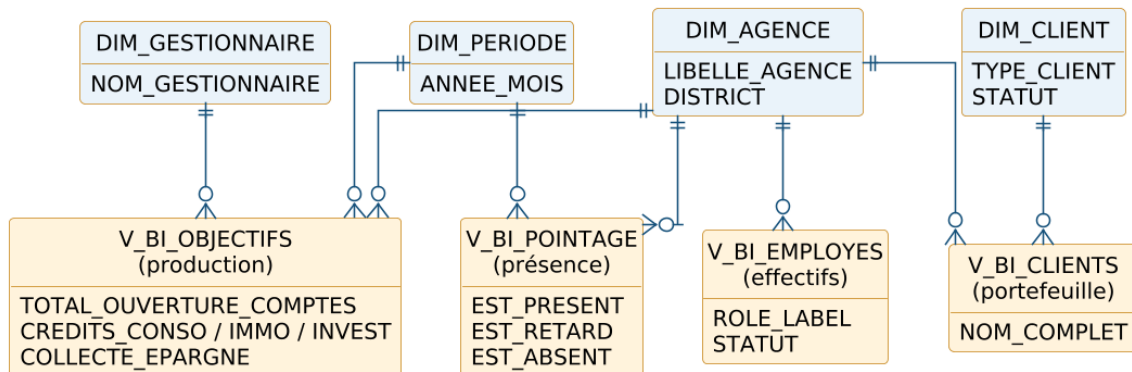


FIGURE 3.10 – Modèle décisionnel en étoile (vues V_BI_*)

Le tableau 3.1 récapitule les principaux indicateurs (KPI) retenus.

TABLE 3.1 – Indicateurs de performance (KPI) du volet décisionnel

Indicateur	Description	Source
Effectifs par agence	Nombre d'employés par agence	V_BI_EMPLOYES
Répartition des rôles	Gestionnaires / conseillers / hors commercial	V_BI_EMPLOYES
Portefeuille clients	Clients par type et par agence	V_BI_CLIENTS
Ouvertures de comptes	Comptes chèques + épargne + courants	V_BI_OBJECTIFS
Production de crédits	Conso, immobilier, investissement	V_BI_OBJECTIFS
Collecte d'épargne	Épargne additionnelle collectée	V_BI_OBJECTIFS
Taux de présence	Présents, retards et absences par agence et période	V_BI_POINTAGE

Conclusion

La conception étant établie — des architectures jusqu'au modèle décisionnel — le chapitre suivant présente la réalisation concrète de l'application.

Chapitre 4

Réalisation

Introduction

Ce chapitre présente les choix technologiques et l’architecture de déploiement, puis les principales interfaces de l’application, le module de pointage et la réalisation du volet décisionnel.

4.1 Choix technologiques

Le choix de **Jakarta EE 10** [1] se justifie par sa robustesse, sa standardisation et son adéquation aux applications d’entreprise : injection de dépendances, gestion transactionnelle déclarative (JTA), persistance via JPA (**Hibernate** [2]) et sécurité intégrée. La base **Oracle** [3] a été retenue pour sa fiabilité et sa large adoption dans le secteur bancaire, et l’application est déployée sur **WildFly** [4]. Pour la restitution décisionnelle, **Power BI** [5] offre une connexion directe aux vues Oracle et des capacités de visualisation avancées, complétées par un tableau de bord embarqué (**ApexCharts** [6]) pour une consultation immédiate au sein de l’application.

4.2 Architecture de déploiement

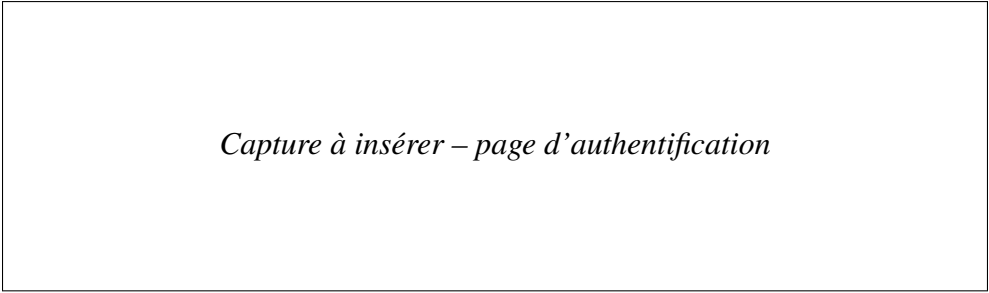
L’application est empaquetée sous forme d’archive WAR (gestion-agences) et déployée sur WildFly sous le contexte /gestion-agences. L’accès à la base s’effectue via une source de données JTA (OracleDS) gérée par le conteneur, conformément à l’architecture physique du chapitre 3.2.

4.3 Présentation des interfaces

Les figures suivantes présentent les principales interfaces de l’application. *Les captures d’écran réelles seront insérées à cet emplacement.*

4.3.1 Page d’authentification

La figure 4.1 présente la page de connexion.

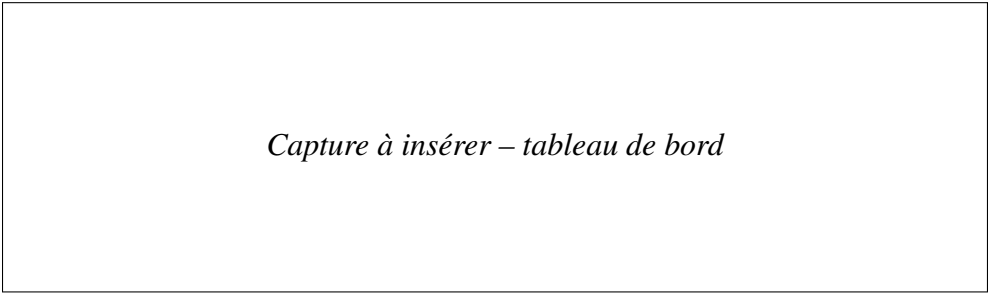


Capture à insérer – page d’authentification

FIGURE 4.1 – Interface d’authentification

4.3.2 Tableau de bord

La figure 4.2 présente le tableau de bord. Il comporte une rangée de **cartes d’indicateurs** (employés, clients, présents du jour, taux de présence, demandes en attente) puis des **graphiques dynamiques** ApexCharts : l’*effectif par agence* (histogramme), la *répartition des rôles* (anneau) et la *présence du jour* (anneau présents / retards / absents). Ces indicateurs offrent aux responsables une vision synthétique et immédiate de l’activité.

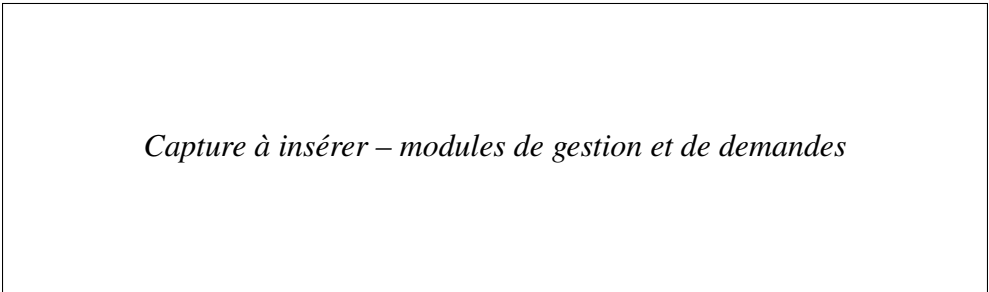


Capture à insérer – tableau de bord

FIGURE 4.2 – Tableau de bord de l’application

4.3.3 Modules de gestion et de demandes

La figure 4.3 illustre les modules de gestion (agences, employés, clients) et le traitement des demandes.



Capture à insérer – modules de gestion et de demandes

FIGURE 4.3 – Modules de gestion et de validation des demandes

4.4 Réalisation du volet décisionnel

Les vues analytiques (V_BI_EMPLOYES, V_BI_CLIENTS, V_BI_OBJECTIFS) ont été créées dans la base Oracle puis connectées à Power BI, où des rapports interactifs analysent les effectifs, le portefeuille clients et la production par agence et par période (figure 4.4).

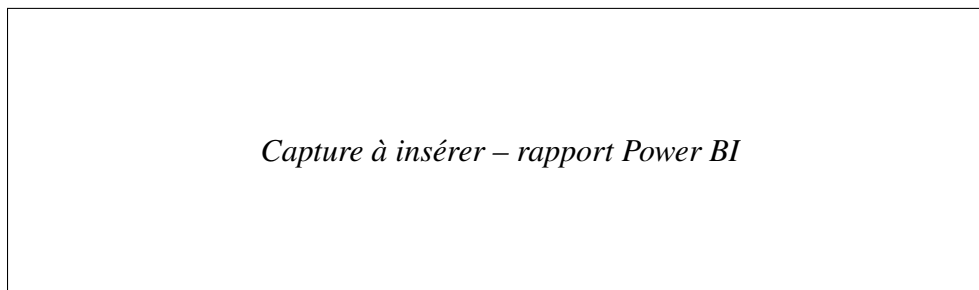


FIGURE 4.4 – Rapport Power BI – analyse des effectifs et des objectifs

4.5 Module de pointage

Le module de pointage assure le suivi de la présence des employés. Chaque employé enregistre son arrivée et son départ via un bouton « Pointer » (l'heure est relevée sur l'heure système); le statut est déterminé automatiquement (PRESENT, ou RETARD au-delà de 9h00). L'administrateur peut en outre saisir ou corriger manuellement un pointage (PRESENT, RETARD ou ABSENT). Techniquement, le module repose sur l'entité Pointage, le service PointageService (EJB) et le servlet PointageServlet (/pointage); il alimente la vue décisionnelle V_BI_POINTAGE (taux de présence, retards, absences par agence et période). La figure 4.5 en présente l'interface.

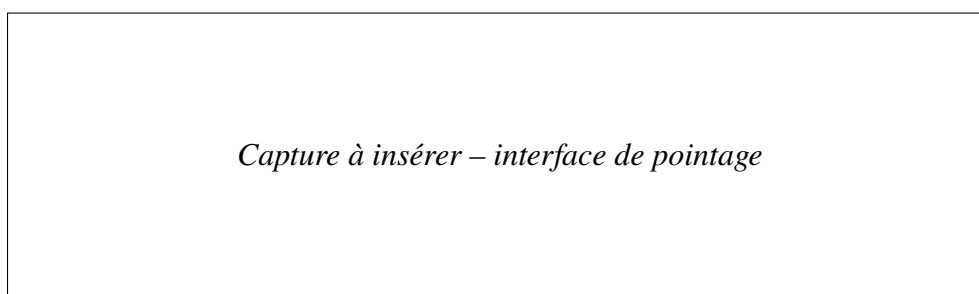


FIGURE 4.5 – Interface du module de pointage

4.6 Sécurité et gestion des rôles

L'accès à l'application est contrôlé par le SecurityService, qui authentifie les utilisateurs à partir de la table dédiée et associe à chacun un rôle (ADMIN, DIRECTEUR_COMMERCIAL, USER). Ce rôle conditionne l'accès aux fonctionnalités. Les opérations sensibles sont encadrées par les workflows de validation, et chaque action significative est journalisée à des fins d'audit.

Conclusion

La réalisation a permis d'aboutir à une application fonctionnelle couvrant les besoins exprimés, complétée par un dispositif décisionnel opérationnel.

Conclusion générale et perspectives

Ce projet de fin d'études a permis de concevoir et de réaliser, au sein de la **BTK – Banque Tuniso-Koweïtienne**, une application web de gestion des agences bancaires adossée à un volet décisionnel. La solution centralise la gestion des agences, des employés, des clients et des objectifs, formalise les opérations sensibles au moyen de circuits de validation, et restitue aux responsables des indicateurs d'aide à la décision via un tableau de bord embarqué et des rapports Power BI.

Sur le plan technique, ce travail a été l'occasion de mettre en pratique les compétences acquises durant la formation : modélisation UML, développement Jakarta EE (Servlets, JSP, EJB, JPA), conception de bases de données Oracle et mise en œuvre d'une chaîne décisionnelle (vues analytiques et reporting Power BI), au croisement du développement logiciel et de la Business Intelligence.

Plusieurs perspectives d'évolution peuvent être envisagées :

- le chiffrement (hachage) des mots de passe et le renforcement de la sécurité ;
- l'automatisation de l'alimentation d'un entrepôt de données (ETL) pour historiser les indicateurs ;
- l'enrichissement par des analyses prédictives (prévision des objectifs) ;
- l'exposition d'une API REST pour l'interopérabilité avec d'autres systèmes ;
- le développement d'une application mobile de consultation des indicateurs.

Nétographie

- [1] Eclipse Foundation, “Jakarta ee 10 – documentation officielle,” 2025. <https://jakarta.ee/>.
- [2] Red Hat, “Hibernate orm – documentation,” 2025. <https://hibernate.org/orm/documentation/>.
- [3] Oracle Corporation, “Oracle database – documentation,” 2025. <https://docs.oracle.com/en/database/>.
- [4] Red Hat, “Wildfly application server – documentation,” 2025. <https://docs.wildfly.org/>.
- [5] Microsoft, “Microsoft power bi – documentation,” 2025. <https://learn.microsoft.com/power-bi/>.
- [6] ApexCharts, “Apexcharts.js – documentation,” 2025. <https://apexcharts.com/docs/>.
- [7] Apache Software Foundation, “Apache maven – documentation,” 2025. <https://maven.apache.org/guides/>.
- [8] BTK Bank, “Banque tuniso-koweïtienne – site officiel,” 2025. <https://www.btknet.com/>.

Annexe A

Annexes

A.1 Scripts SQL d'initialisation

Les scripts de création des tables et des séquences (sécurité, demandes, modifications) ainsi que les définitions des vues décisionnelles (V_BI_EMPLOYES, V_BI_CLIENTS, V_BI_OBJECTIFS) figurent dans le dépôt du projet, sous le répertoire `sql/`.

A.2 Captures complémentaires

Des captures d'écran complémentaires de l'application et des rapports Power BI pourront être insérées dans cette annexe.

A.3 Dépôt du code source

Le code source complet du projet est disponible sur le dépôt GitHub associé au rapport.